

INFORME TÉCNICO
Taller CRUD - Ejercicio 10
Cooperativa de Caficultores "Café de Altura"

Aprendiz: Samuel Moreno Carmona

Instructor: Richard Betancur

Fecha: junio 2026

1. Introducción

El presente informe documenta el desarrollo del Ejercicio 10 del taller CRUD, correspondiente al sistema de gestión para la Cooperativa de Caficultores "Café de Altura". El sistema fue implementado como una API REST utilizando Node.js y Express, con persistencia de datos en archivos JSON.

El objetivo principal fue implementar las operaciones CRUD sobre las entidades del negocio (Asociado, Finca, Cosecha y Venta), aplicando validaciones de integridad referencial y pruebas funcionales que garantizan el correcto funcionamiento del sistema.

2. Requerimientos Mínimos

2.1 Requerimientos Funcionales

- Registrar nuevos asociados a la cooperativa con sus datos básicos.
- Consultar asociados filtrando por estado: activo o inactivo.
- Actualizar datos de asociados y sus fincas registradas.
- Eliminar un asociado únicamente si no tiene fincas ni cosechas registradas.
- Registrar fincas vinculadas a un asociado propietario.
- Registrar cosechas por finca y temporada con control de inventario.
- Registrar ventas de café descontando la cantidad vendida del saldo de cosecha.
- Validar que no se venda más cantidad de la disponible en inventario.

2.2 Requerimientos No Funcionales

- API REST desarrollada en Node.js con Express.
- Persistencia de datos mediante archivos JSON.
- Pruebas funcionales automatizadas con Jest.
- Código organizado en capas: modelos, servicios y rutas.

3. Diseño de la Solución

3.1 Arquitectura del Sistema

El sistema sigue una arquitectura en capas separando responsabilidades:

Capa	Descripción
Modelos	Clases JavaScript que definen la estructura de cada entidad.
Servicios	Lógica de negocio, validaciones y acceso a los archivos JSON.
Rutas	Endpoints HTTP que reciben peticiones y delegan a los servicios.
Datos	Archivos JSON que actúan como base de datos del sistema.

3.2 Entidades del Sistema:

Entidad	Campos Principales	Observaciones
Asociado	identificacion, nombre, telefono, aniosCaficultor, numeroDeFincas, estado	Estado: activo / inactivo. numeroDeFincas se actualiza automáticamente.
Finca	codigo, nombre, ubicacion, areaHectareas, variedadesCafe, asociadold	Vinculada a un asociado propietario mediante asociadold.
Cosecha	numero, fincaCodigo, temporada, variedadCafe, cantidadProducida, saldoKg, fechaRecoleccion, estado	saldoKg inicia igual a cantidadProducida. Estado: en proceso / recolectada / en beneficio / terminada.
Venta	numeroVenta, cosechaNumero, comprador, fecha, cantidadVendida, precioKg, total	Al registrar una venta se descuenta cantidadVendida del saldoKg de la cosecha.

3.3 Endpoints Implementados

Método	Endpoint	Descripción
GET	/asociados	Listar todos o filtrar por ?estado=activo/inactivo
POST	/asociados	Registrar nuevo asociado
PUT	/asociados/:id	Actualizar datos de un asociado
DELETE	/asociados/:id	Eliminar asociado (solo si no tiene registros vinculados)
GET	/fincas	Listar todas las fincas
POST	/fincas	Registrar nueva finca
PUT	/fincas/:codigo	Actualizar datos de una finca
DELETE	/fincas/:codigo	Eliminar una finca
GET	/cosechas	Listar todas las cosechas
POST	/cosechas	Registrar nueva cosecha
PUT	/cosechas/:numero	Actualizar datos de una cosecha
DELETE	/cosechas/:numero	Eliminar una cosecha
GET	/ventas	Listar todas las ventas
POST	/ventas	Registrar nueva venta (descuenta inventario)

4. Implementación

4.1 Estructura de Carpetas:

Archivo / Carpeta	Descripción
server.js	Punto de entrada. Configura Express y registra las rutas.
src/models/	Clases Asociado, Finca, Cosecha y Venta.
src/services/	Lógica de negocio y acceso a datos para cada entidad.
src/routes/	Definición de endpoints HTTP por entidad.
src/data/	Archivos JSON: asociados, fincas, cosechas y ventas.
test/pruebas.test.js	Pruebas funcionales automatizadas con Jest.

4.2 Lógica de Negocio Destacada

Validación al eliminar asociado:

Antes de eliminar un asociado, el sistema verifica en dos pasos: primero que no tenga fincas registradas, y luego que no tenga cosechas vinculadas a sus fincas. Si alguna condición falla, se retorna un error HTTP 400 con mensaje descriptivo.

Control de inventario en ventas:

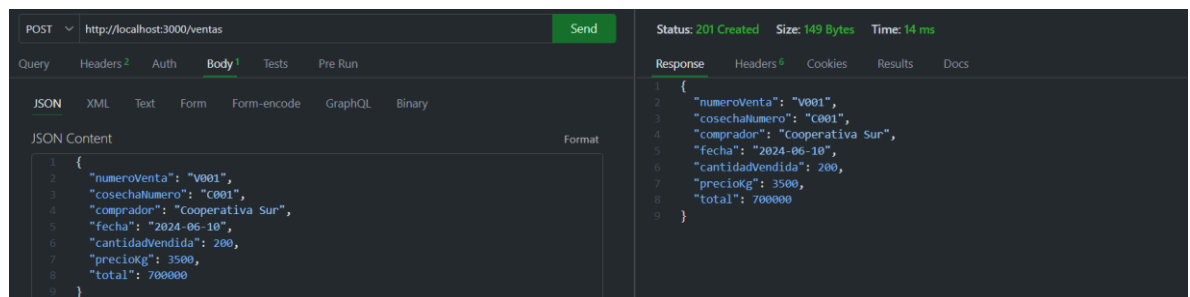
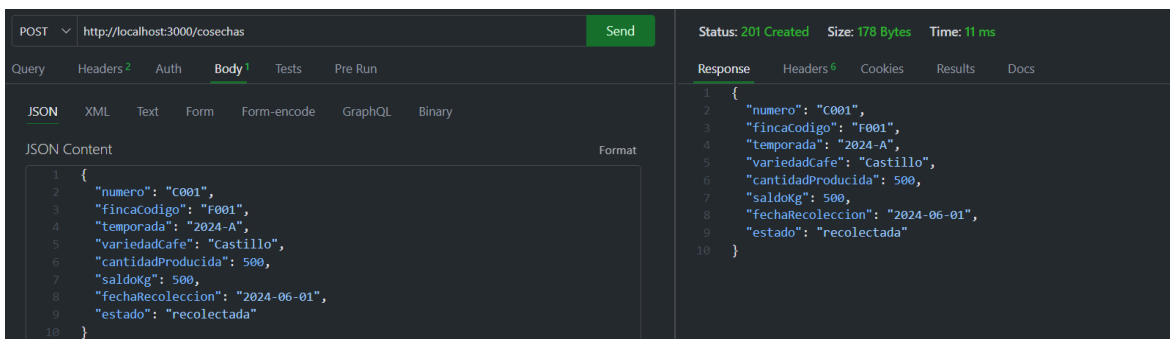
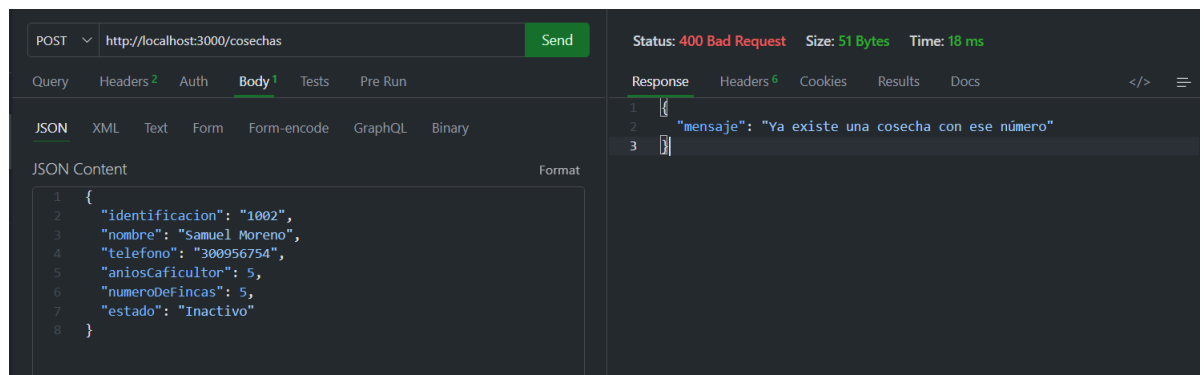
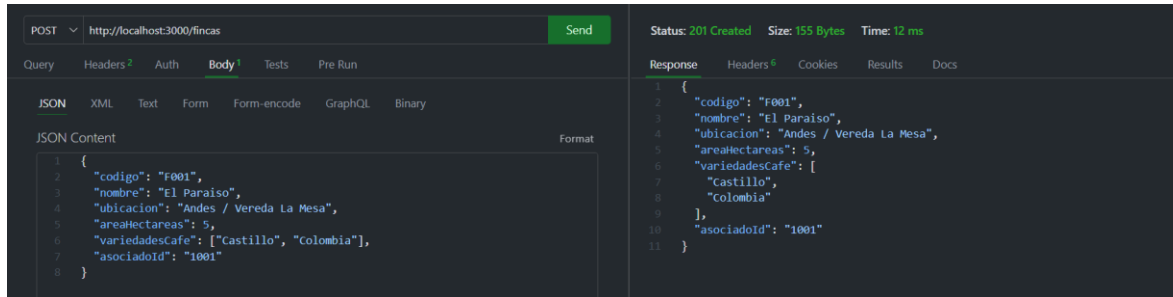
Al registrar una venta, el sistema valida que la cantidad vendida no supere el saldoKg disponible en la cosecha. Si la validación es exitosa, descuenta la cantidad del saldo y guarda ambos registros de forma atómica.

Contador automático de fincas:

El campo numeroDeFincas del asociado se incrementa automáticamente al crear una finca y se decrementa al eliminarla, manteniendo siempre el conteo actualizado.

4.3 Evidencias de Implementación

Capturas sugeridas: servidor corriendo, peticiones en Thunder Client o Postman, terminal con los tests pasando.



GET

http://localhost:3000/cosechas/C001

Send

Query

Headers 2

Auth

Body

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

Binary File

Choose File

Select file

Status: 200 OK

Size: 178 Bytes

Time: 9 ms

Response

Headers 6

Cookies

Results

Docs

1

{

2

"numero": "C001",

3

"fincaCodigo": "F001",

4

"temporada": "2024-A",

5

"variedadCafe": "Castillo",

6

"cantidadProducida": 500,

7

"saldokg": 300,

8

"fechaRecoleccion": "2024-06-01",

9

"estado": "recolectada"

10

}

DELETE

http://localhost:3000/asociados/1002

Send

Query

Headers 2

Auth

Body

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

Binary File

Choose File

Select file

Status: 400 Bad Request

Size: 72 Bytes

Time: 7 ms

Response

Headers 6

Cookies

Results

Docs

1

{

2

"mensaje": "No se puede eliminar: el asociado tiene fincas registradas"

3

}

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\USUARIO\OneDrive\Documentos\proyecto_caficultores> npm test

> proyecto_caficultores@1.0.0 test

> jest

PASS __test__pruebas.test.js

✓ Prueba 1: flujo completo de venta y verificacion de saldo (4 ms)

✓ Prueba 2: eliminar asociado solo cuando no tiene registros vinculados (11 ms)

Test Suites: 1 passed, 1 total

Tests: 2 passed, 2 total

Snapshots: 0 total

Time: 0.247 s, estimated 1 s

Ran all test suites.

PS C:\Users\USUARIO\OneDrive\Documentos\proyecto_caficultores>

5. Pruebas Funcionales

5.1 Prueba 1 — Flujo completo de venta y verificación de saldo

Campo	Detalle
Descripción	Registrar un asociado, crear una finca a su nombre, registrar una cosecha de 500 kg, realizar una venta de 200 kg y verificar que el saldo quede en 300 kg.
Datos de entrada	Asociado ID: 1001, Finca: F001, Cosecha: C001 de 500 kg, Venta: V001 de 200 kg a \$3.500/kg
Resultado esperado	cosecha.saldoKg === 300
Resultado obtenido	cosecha.saldoKg === 300
Estado	APROBADO

5.2 Prueba 2 — Validación de eliminación con registros vinculados

Campo	Detalle
Descripción	Intentar eliminar un asociado que tiene fincas y cosechas registradas. Verificar que el sistema impide la eliminación en cada caso y solo la permite cuando no existen registros vinculados.
Datos de entrada	Asociado ID: 2001, Finca: F002, Cosecha: C002 de 300 kg
Resultado esperado	Intento 1 (con finca): error. Intento 2 (sin finca, pero con cosecha): cosecha definida. Intento 3 (sin registros): eliminación exitosa.
Resultado obtenido	Todos los escenarios respondieron según lo esperado.
Estado	APROBADO

5.3 Resultado General

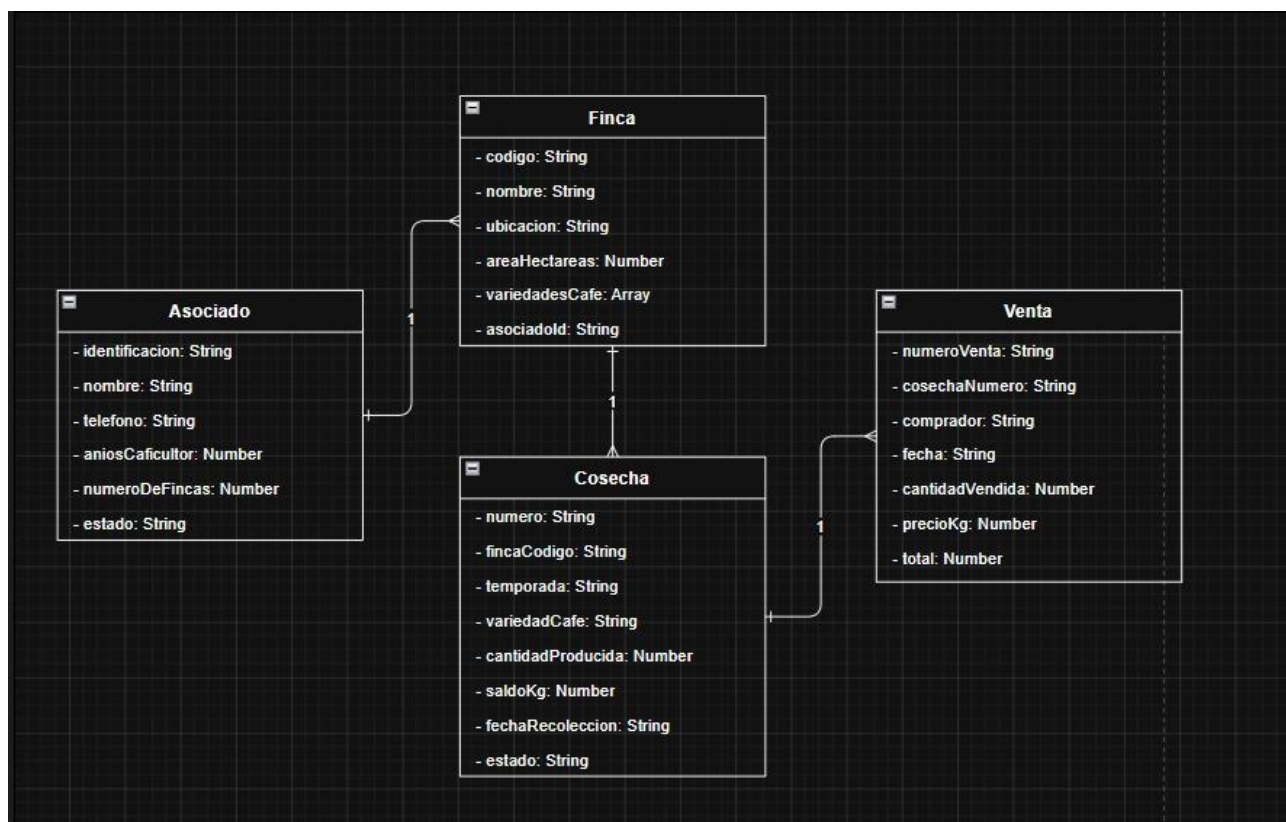
N°	Prueba	Tiempo	Estado
1	Flujo completo de venta y verificación de saldo	5 ms	APROBADO
2	Eliminación de asociado con validación de registros vinculados	10 ms	APROBADO

Test Suites: 1 passed, 1 total | Tests: 2 passed, 2 total | Tiempo total: 0.354 s

6. Diagrama de Clases

El diagrama de clases ilustra las cuatro entidades principales del sistema y sus relaciones:

- Asociado tiene una relación de composición con Finca (un asociado puede tener múltiples fincas).
- Finca tiene una relación de composición con Cosecha (una finca puede tener múltiples cosechas por temporada).
- Cosecha tiene una relación de composición con Venta (una cosecha puede tener múltiples ventas).



7. Conclusiones

- Se implementaron exitosamente las cuatro operaciones CRUD sobre las entidades Asociado, Finca, Cosecha y Venta, cumpliendo con todos los requerimientos del enunciado.
- La validación de integridad referencial al eliminar asociados garantiza la consistencia de los datos, impidiendo eliminar registros con dependencias activas.
- El control de inventario en las ventas funciona correctamente, descontando el saldo de la cosecha en cada transacción.
- Las pruebas funcionales automatizadas con Jest confirmaron el correcto funcionamiento de los escenarios críticos, aprobando los 2 casos de prueba en 0.354 segundos.
- La arquitectura en capas facilita el mantenimiento y la escalabilidad del sistema.

8. Recomendaciones

- Migrar la persistencia de archivos JSON a una base de datos relacional como PostgreSQL o MySQL para mejorar el rendimiento en producción.
- Implementar autenticación y autorización mediante JWT para proteger los endpoints de la API.
- Ampliar la cobertura de pruebas incluyendo casos de error para robustecer el sistema.
- Agregar validación de esquema en las rutas para verificar que los datos de entrada cumplan el formato requerido.
- Considerar el uso de variables de entorno para configurar el puerto y las rutas de los archivos de datos.